Цель работы

Изучить механизмы динамического выделения и освобождения памяти в языке C++, освоить работу с операторами new/delete и функциями malloc/free, научиться предотвращать утечки памяти и корректно управлять ресурсами.

Краткая теория

Динамическая память - память выделяемая в процессе выполнения программмы (<<в куче>>). В C базовым способом является malloc/free, в C++ - new/delete.

При выделении и не освобождении памяти может произойти утечка памяти, что приводит к потреблению излишних ресурсов, а иногда и к падению программы.

Задания

задание 1

```
#include <iostream>
using namespace std;

int main()
{
  //выделяем память
  int *p = new(nothrow) int;

  //проверка
  if (p==nullptr)
  {
    cerr << "error with memory";
    return 1;
  }

  //запрашиваем число
  cout << "Введите число для хранения\n"; cin >> *p;

  //выводим
  cout << "ваше число --- " << *p << endl;
  //очищаем
  delete p;

  return 0;
}
```

```
➜  src ../bin/laba11
Введите число для хранения
5
ваше число —— 5
➜  src
```

задание 2

```
#include <cstdlib>
#include <ctime>
#include <iostream>
using namespace std;

int main()
{
  int size;
  cout << "введите желаемый размер массива:"; cin >> size;

  if (size <= 0)
  {
    cerr << "невозможный размер массива";
    return 1;
  }

  int *p =new(nothrow) int[size];

  if (p==nullptr)
  {
    cerr << "слишком большой массив";
    return 2;
  }

  srand(time(NULL));
  for (int q=0; q < size; q++)
  {
    p[q] = rand()%100;
  }

  for (int i=0; i < size; i++)
  {
    cout << p[i] << " ";
  }

  delete[] p;

  return 0;

}
```

```
→  src ../bin/laba12
введите желаемый размер массива:5
79 1 30 88 76 ⏎
→  src
```

задание 3

```
#include <cstdlib>
#include <ctime>
#include <iostream>
using namespace std;

int main()
{
  int size;
  cout << "Введите желаемый размер массива:"; cin >> size;

  if (size <= 0)
  {
    cout << "невозможный размер массива";
    return 1;
  }

  int *p = (int*)malloc(10 * sizeof(int));
  if (p == nullptr)
  {
    cerr << "ошибка выделения памяти";
  }

  srand(time(NULL));
  for (int q=0; q < size; q++)
  {
    p[q] = rand()%100;
  }

  for (int i=0; i < size; i++)
  {
    cout << p[i] << " ";
  }

  free(p);
}
```

```
➜  src ../bin/laba13
Введите желаемый размер массива:15
89 44 13 63 25 46 53 94 16 85 75 0 41 77 76 ⏎
➜  src
```

задание 4

```cpp
#include <iostream>


int main()
{
  int str=0;int stlb=0;

  std::cout << "введите желаемый размер матрицы in str:";
  std::cin >> str;

  std::cout << std::endl;

  std::cout << "введите желаемый размер матрицы in stlb:";
  std::cin >> stlb;

  if (str <= 0 || stlb <= 0 ){
    std::cerr << "невозможный размер матрицы";
    return 1;
  }

  std::cout << "create matrix" << std::endl;
  int** matrix = new int*[str];
  std::cout << "matrix created" << std::endl;

  for (int q =0; q < str; q++)
  {
    matrix[q] = new int[stlb];
  }

  for (int strok = 0; strok < str; strok++)
  {
    for (int stolb = 0; stolb < stlb; stolb++)
    {
      matrix[strok][stolb] = 0;
    }
  }

  for (int strok = 0; strok < str; strok++)
  {
    for (int stolb = 0; stolb < stlb; stolb++)
    {
      std::cout << matrix[strok][stolb] << " ";
    }
    std::cout << std::endl;
  }
```

```
  for (int q =0; q < str; q++)
  {
    delete[] matrix[q];
  }
  delete[] matrix;
  return 0;
}
```

```
→  src ../bin/laba14
введите желаемый размер матрицы in str:7 7

введите желаемый размер матрицы in stlb:create matrix
matrix created
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
→  src
```

Задание 5

```
#include <cstring>
#include <iostream>
#include <string.h>

struct Students{
  int age;
  char * name;
  double grade;
};

int main(int argc, char* argv[])
{
  int N = std::stoi(argv[1]);

  Students* studs = new Students[N];

  for (int i=0; i < N; i++)
  {
    std::cout << "student №" << i+1 << std::endl;
    char buffer[100];

    std::cout<<"name:";
    std::cin>>buffer;
    int namelen = std::strlen(buffer);

    studs[i].name = new (std::nothrow)char[namelen+1];
    if (studs[i].name == nullptr)
    {
      std::cerr << "memory error";
      return 1;
    }
    std::strcpy(studs[i].name, buffer);
    std::cout << "age:";
    std::cin >> studs[i].age;

    std::cout << "grade:";
    std::cin >> studs[i].grade;

    std::cout<<std::endl;
  }

  for (int i=0; i < N; i++)
  {
    std::cout << "student №" << i+1 << std::endl;
    std::cout << "stud name" << studs[i].name << '\n';
```

```
    std::cout << "age" << studs[i].age << '\n';
    std::cout << "stud grade" << studs[i].grade << '\n';
    std::cout<<std::endl;
  }

  for (int i = 0; i < N; ++i){
    delete[] studs[i].name;
    studs[i].name=nullptr;
  }

  delete[] studs;

  return 0;
}
```

```
→  src ../bin/laba15 3
student №1
name:Kirill
age:18
grade:4.3

student №2
name:Yarik
age:18
grade:4.8

student №3
name:Dima
age:17
grade:3.2

student №1
stud nameKirill
age18
stud grade4.3

student №2
stud nameYarik
age18
stud grade4.8

student №3
stud nameDima
age17
stud grade3.2

→  src
```

Контрольные вопросы

1. В чём отличие статического и динамического выделения памяти

статическое	**динамическое**
количество памяти известно на этапе компиляции	количество памяти выделяется в процессе
автоматическая работа с памятью	ручное управление памятью

2. Что возвращает оператор new при неудаче? Как это обработать?

   Он возвращает bad_alloc.

   Для обработки следует использовать std::nothrow и проверять на nullptr.

3. Почему важно освобождать память через delete/free?

   Иначе будет заниматься слишком много памяти, что скажется на быстродействии и вероятности краша.

4. Чем отличаются new/delete от malloc/free?

   new сам считает размер основываясь на типе величины, mallo получает память в байтах.

5. Что такое утечка памяти и как её избежать?

   Утечка памяти - большое кол-во памяти используется ненужными данными, следует очищать память через delete.

6. Как выделить память для двумерного массива?

   Выделить массив одномерных указателей

7. Почему при освобождении двумерного массива нужно сначала удалять строки?

   Потому что иначе к этим строкам не будет доступа и получится утечка памяти без возможности решения.

Выводы

Узнал о работе с динамической памятью и структурами в Cpp и разнице работы с памятью между C и Cpp